

---

Process Management

# Synchronization

---

Lecture 6 (**Live** Version)

# Overview

## ■ Race Condition

- ❑ Problems with concurrent execution

## ■ Critical Section

- ❑ Properties of correct implementation
- ❑ Symptoms of incorrect implementation

## ■ Implementations of Critical Section

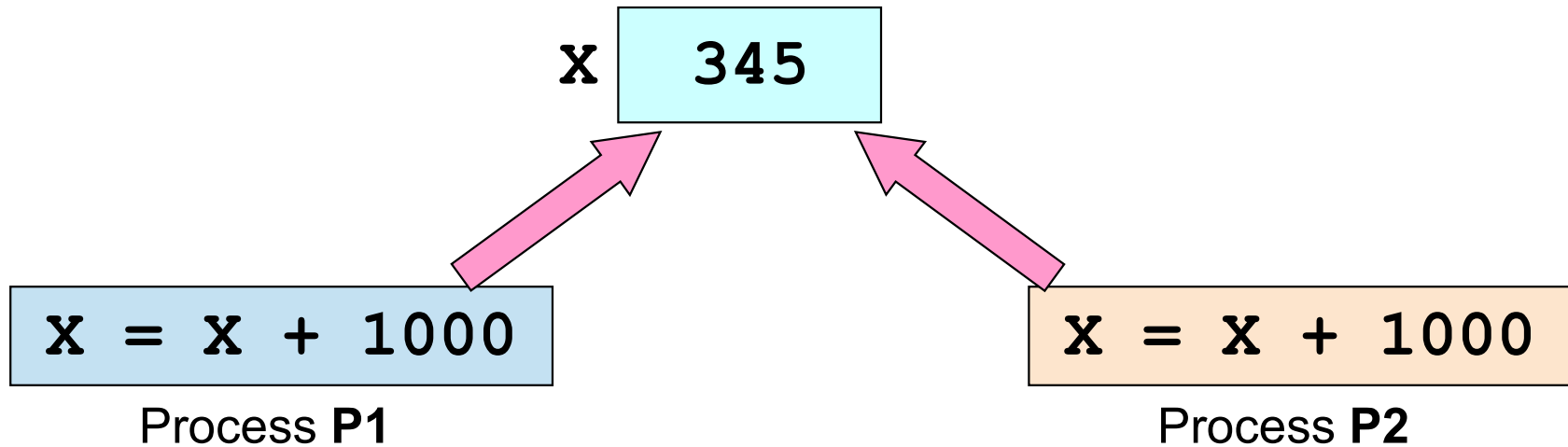
- ❑ Low level
- ❑ High level language
- ❑ High level abstraction

## ■ Classical synchronization problems

# Problems with Concurrent Execution

- When two or more processes:
  - ❑ Execute concurrently in interleaving fashion
  - ❑ Share a modifiable resource
  - Can cause **synchronization problems**
- Execution of a single sequential process is **deterministic**
  - ❑ Repeated execution gives the same result
- Execution of concurrent processes may be **non-deterministic**
  - ❑ Execution outcome depends on the order in which the shared resource is access/modified
  - ❑ known as **race conditions**

# Race Condition: Illustration



- Process P1 and P2 shares a variable **x**
- Rough translation for **x = x + 1000**
  1. **Load** x → Register1
  2. **Add** 1000 to Register1
  3. **Store** Register1 → x

# Race Condition: Round 1

| Time | Value of X | P1 | P2 |
|------|------------|----|----|
| 1    | 345        |    |    |

## Race Condition: Round 2

| Time | Value of X | P1               | P2 |
|------|------------|------------------|----|
| 1    | 345        | Load X → Reg1    |    |
| 2    | 345        | Add 1000 to Reg1 |    |

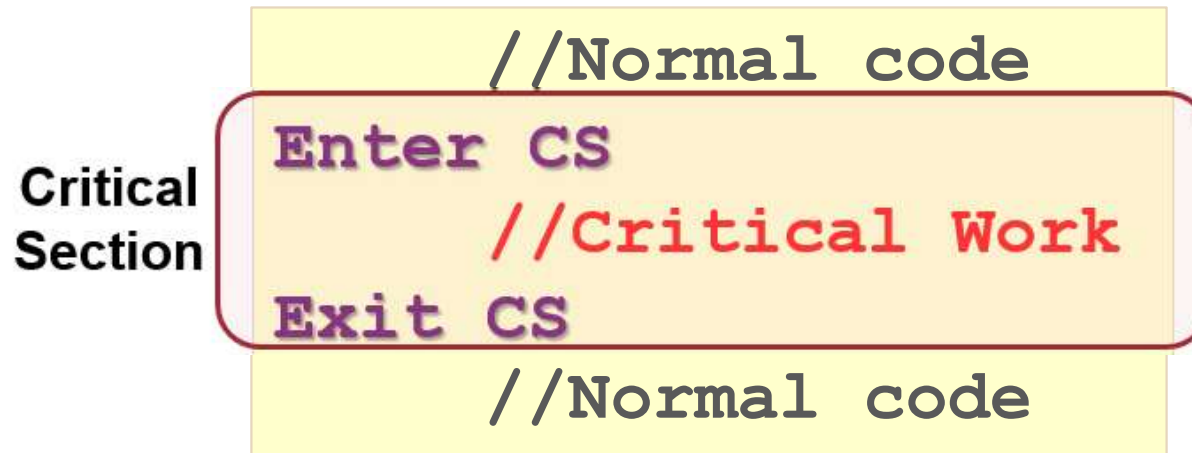
- When two or more processes:
  - ❑ Execute concurrently in interleaving fashion
  - ❑ Share a modifiable resource
  - Can cause **synchronization problems**

## Race Condition: **Solution**

Incorrect execution is due to the **unsynchronized access to a shared modifiable resource**

- Solution outline:
  - ❑ Designate code segment with race condition as **critical section**
  - ❑ At any point in time, only **one process** can execute in the critical section

# Critical Section (CS)



```
Enter CS
    X = X + 1000
Exit CS
```

Process P1

```
Enter CS
    X = X + 1000
Exit CS
```

Process P2

# Properties of Correct CS Implementation

## Mutual Exclusion:

- If process  $P_i$  is executing in critical section, all other processes are prevented from entering the critical section.

## Progress:

- If no process is in a critical section, one of the waiting processes should be granted access.

## Bounded Wait:

- After process  $P_i$  request to enter critical section, there exists an upperbound of number of times other processes can enter the critical section before  $P_i$ .

## Independence:

- Process **not** executing in critical section should never block other process.

# Symptoms of **Incorrect Synchronization**

## ■ **Deadlock:**

- ❑ All processes blocked → *no progress*

## ■ **Livelock:**

- ❑ Usually related to *deadlock avoidance* mechanism
- ❑ Processes keep changing state to avoid deadlock and make no other progress
- ❑ Typically process are not blocked

## ■ **Starvation:**

- ❑ Some processes are blocked forever

---

# CS Implementations Overview

- **Assembly level implementations:**
  - ❑ Mechanisms provided by the processor
- **High level abstractions:**
  - ❑ Provide abstracted mechanisms that provide additional useful features
  - ❑ Commonly implemented by assembly level mechanisms

---

Don't worry! The processor has all the answers!

# **ASSEMBLY LEVEL IMPLEMENTATION**

# Test and Set: An Atomic Instruction

- Machine instruction to aid synchronization

`TestAndSet Register, MemoryLocation`

- **Behavior:**

1. Load the current content at **MemoryLocation** into **Register**
  2. Stores a **1** into **MemoryLocation**
- ❑ Important: The above is performed as a **single machine operation**, i.e. **atomic**

# Using Test and Set

```
void EnterCS( int* Lock )
{
    while( TestAndSet( Lock ) == 1 ) ;
}
```

\$reg



```
EnterCS ()
    X = X + 1000
ExitCS ()
```

Process P0

\$reg



```
EnterCS ()
    X = X + 1000
ExitCS ()
```

Process P1

Lock



Mutual  
Exclusion:

Progress:

Bounded Wait:

Independence:

```
void ExitCS( int* Lock )
{
    *Lock = 0;
}
```

# Observations and Comments

- The implementation **works!**
  - ❑ However, it employs **busy waiting**
    - ➔ Wasteful use of processing power
- Variants of this instruction exists on most processors:
  - ❑ **Compare and Exchange**
  - ❑ **Atomic Swap**
  - ❑ **Load Link / Store Conditional**

# **HIGH LEVEL ABSTRACTION**



## ion Mechanism

One does not simply  
Btw, I'm **Dutch**.

processes

one or more sleeping

**extra** in 1965

□ **Wait( S )**

■ If **S** ≤ 0, **blocks** (go to sleep)

■ **Decrement S**

□ Can be initialized to any non-negative values initially

□ **Signal( S )**

■ **Increments S**

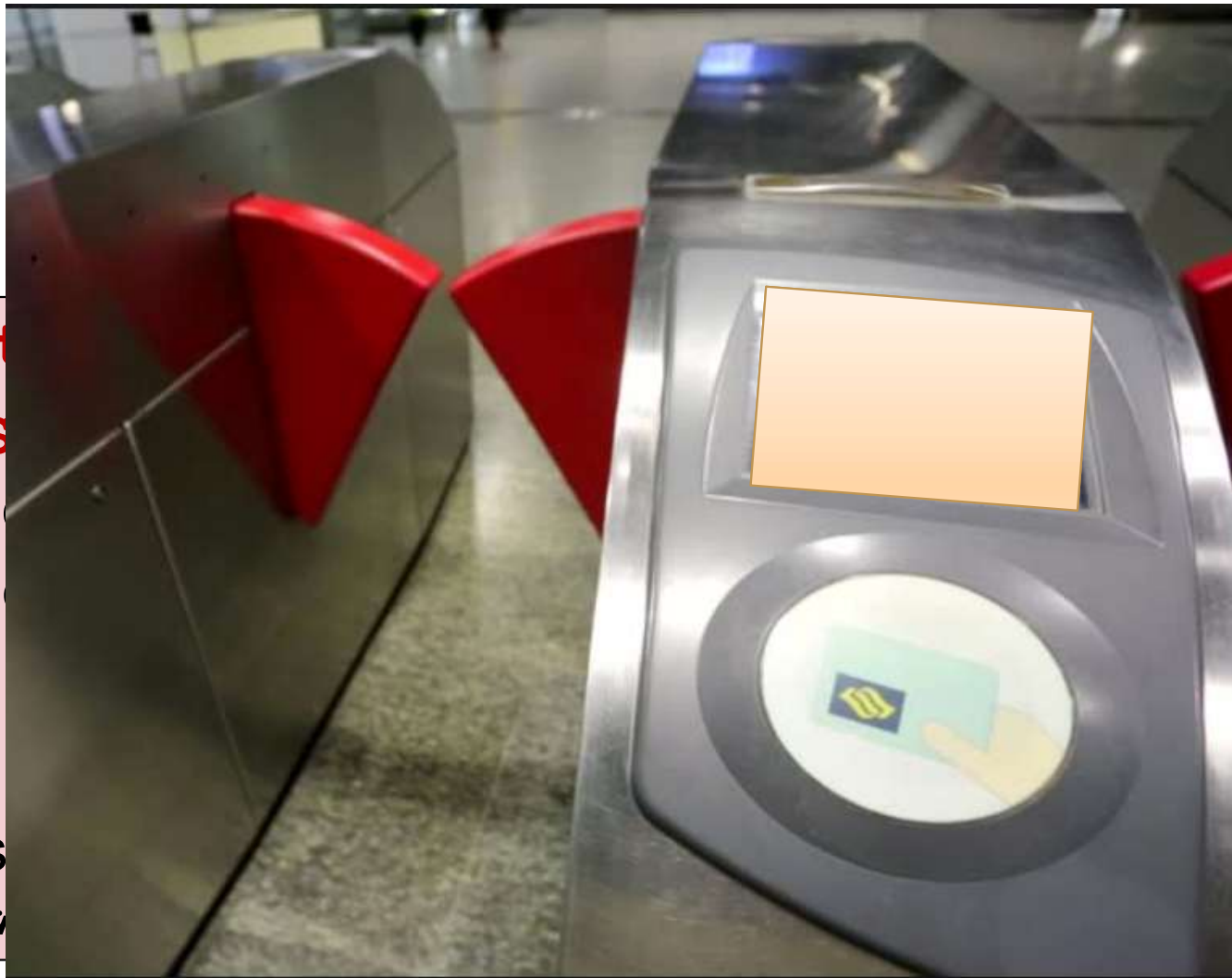
■ Wakes up one sleeping process if any

□ **Wait**

■ If **S** ≤ 0  
sleep

■ **De**

■ Also  
Dow



sleeping

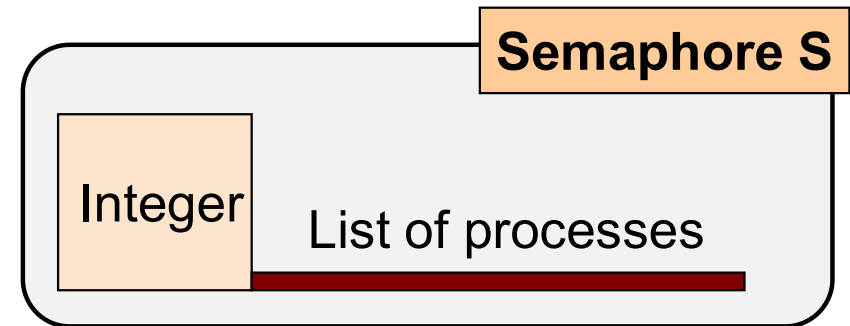
ever

( ) or

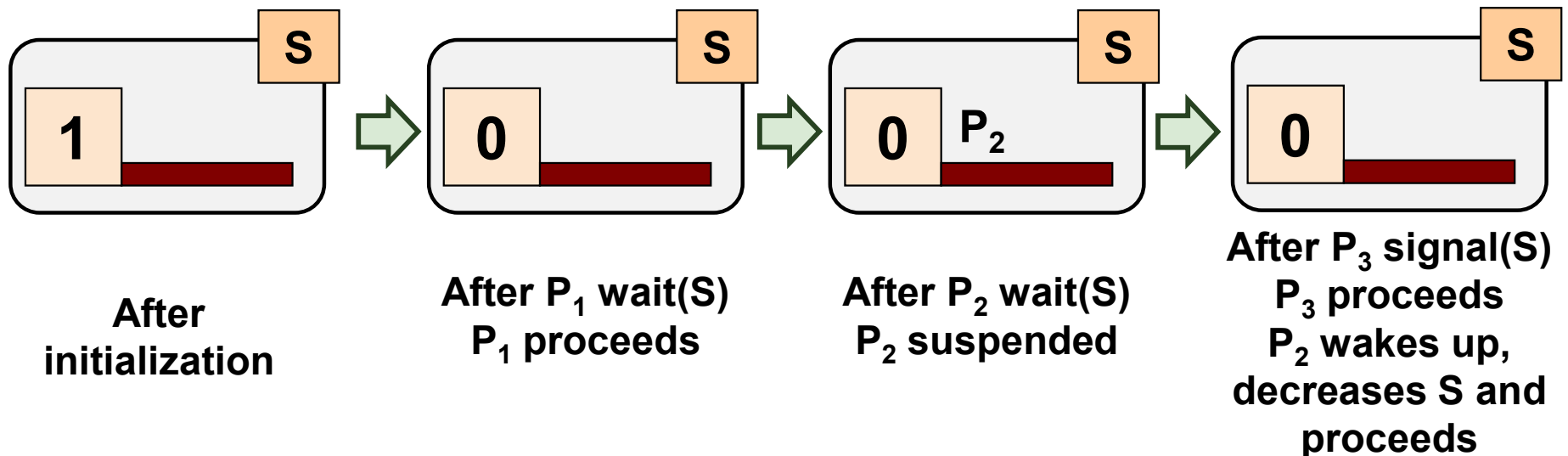
# Semaphore: **Visualization**

- To aid understanding, you can visualize semaphore as:

- ❑ A protected integer
- ❑ A list to keep track of waiting processes



- Example:

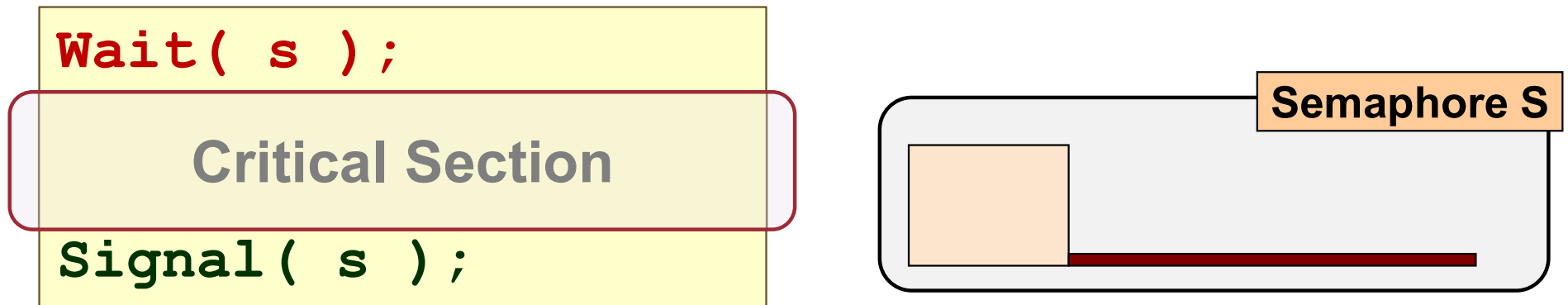


# General and Binary Semaphores

- **General semaphore S:**
  - ❑  $S \geq 0$  ( $S = 0, 1, 2, 3, \dots$ )
  - ❑ also called **counting semaphores**
- **Binary semaphore S:**
  - ❑  $S = 0$  or  $1$
- General semaphore is provided for convenience
  - ❑ **Binary semaphore is sufficient**
  - ❑ i.e. general semaphore can be mimicked by binary semaphores

# Semaphore Example: **Critical Section**

- Binary semaphore **S = 1**



- In this case, **S** can only be 0 or 1
- This semaphore is commonly known as **mutex** (mutual exclusion)

# Incorrect Use of Semaphore: **Deadlock**

- Deadlock is possible with **incorrect use of semaphore**
- Example: Semaphore **P = 1**, **Q = 1** initially

```
Wait( P )  
Wait( Q )
```

Some Code

```
Signal( Q )  
Signal( P )
```

Process P0

```
Wait( Q )  
Wait( P )
```

Some Code

```
Signal( P )  
Signal( Q )
```

Process P1

# Other High Level Abstractions

- Semaphore is **very powerful**:
  - ❑ There are no known unsolvable synchronization problem with semaphore (so far 😊)
  - ❑ Other high level abstractions essentially provide extended features
- Common alternative: **Conditional Variable**
  - ❑ Allow a task to wait for certain event to happens
  - ❑ Has the ability to ***broadcast***, i.e. wakes up all waiting tasks
  - ❑ related to **monitor**

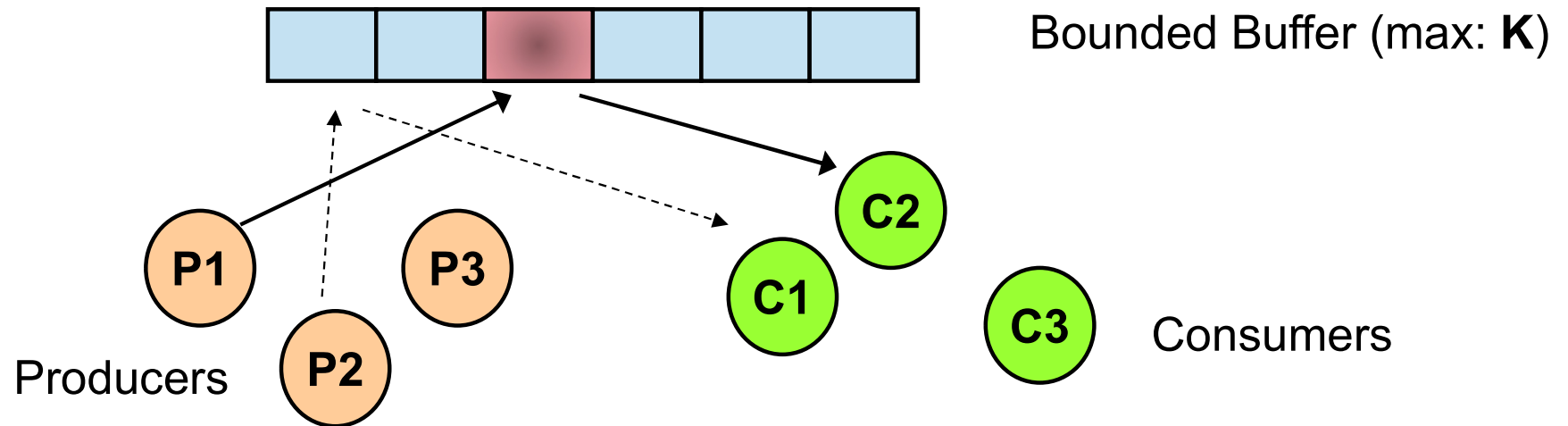
---

Killing brain cells of generations of students.....

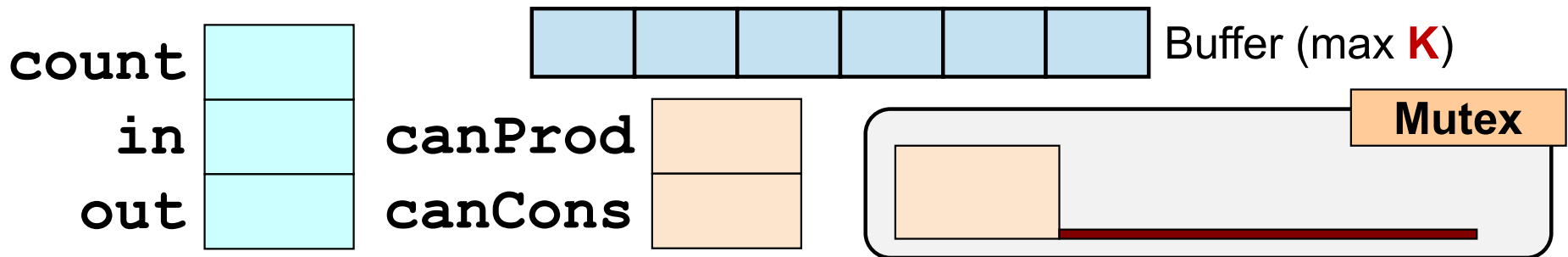
**CLASSICAL**  
**SYNCHRONIZATION**  
**PROBLEMS**

# Producer Consumer: Specification

- Processes share a bounded buffer of size **K**
  - **Producers** produce items to insert in buffer
    - Only when the buffer is **not full** ( $< K$  items)
  - **Consumers** remove items from buffer
    - Only when the buffer is **not empty** ( $> 0$  items)



# Producer Consumer: **Busy Waiting**



```
while (TRUE) {  
  
    Produce Item;  
    if (count < K) {  
        buffer[in] = item;  
        in = (in+1) % K;  
        count++;  
    }  
}
```

Producer

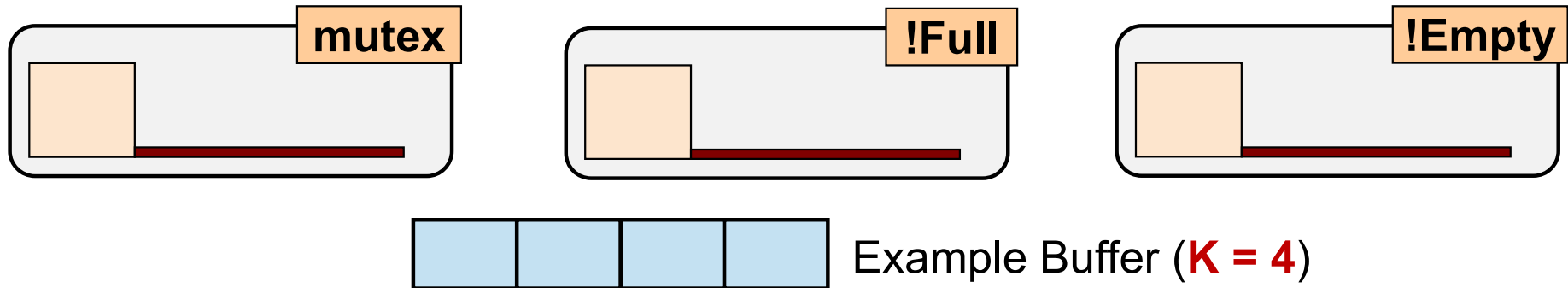
```
while (TRUE) {  
  
    if (count > 0) {  
        item = buffer[out];  
        out = (out+1) % K;  
        count--;  
    }  
  
    Consume Item;  
}
```

Consumer

# Producer Consumer: **Busy Waiting**

- **canConsume:**
  - ❑ Triggers consumer to *try* to get item
- **canProduce:**
  - ❑ Triggers producer to *try* to produce item
- **wait(mutex) + signal(mutex)** : Creates a CS
- **in = (in+1) % K :**  
**out = (out+1) % K :** Wraps around, circular array
- **Evaluation:**
  - ❑ The code **correctly solve** the problem
  - ❑ However, **busy-waiting** is used

# Producer Consumer: **Blocking Version**



```
while (TRUE) {  
    Produce Item;  
  
    wait( notFull );  
    wait( mutex );  
    buffer[in] = item;  
    in = (in+1) % K;  
    count++;  
    signal( mutex );  
    signal( notEmpty );  
}
```

Producer

```
while (TRUE) {  
  
    wait( notEmpty );  
    wait( mutex );  
    item = buffer[out];  
    out = (out+1) % K;  
    count--;  
    signal( mutex );  
    signal( notFull );  
  
    Consume Item;  
}
```

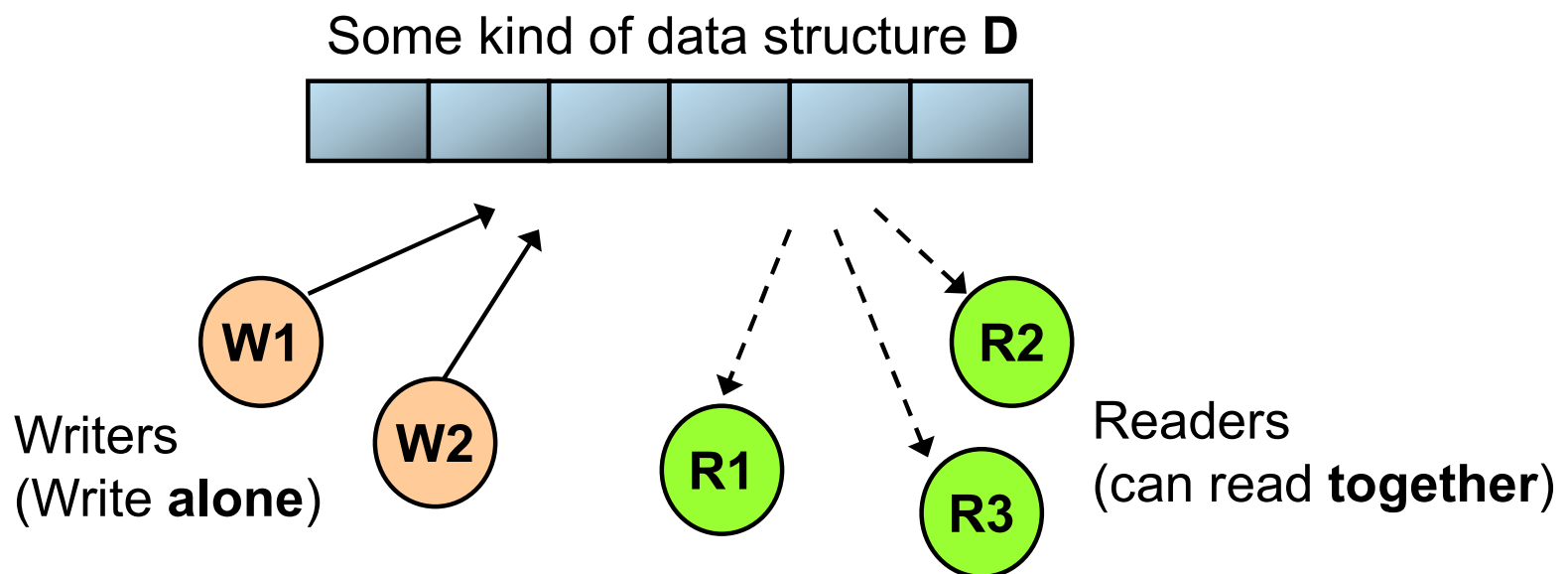
Consumer

# Producer Consumer: **Blocking Version**

- `wait( notFull )` : Forces producers to go to sleep
- `wait( notEmpty )` : Forces consumers to go to sleep
- `signal( notFull )` : 1 consumer wakes up 1 producer
- `signal( notEmpty )` : 1 producer wakes up 1 consumer
- Evaluation:
  - ❑ This code correctly solve the problem
  - ❑ No busy-waiting, “unwanted” producer/consumer will go to sleep on respective semaphores

# Readers Writers: Specification

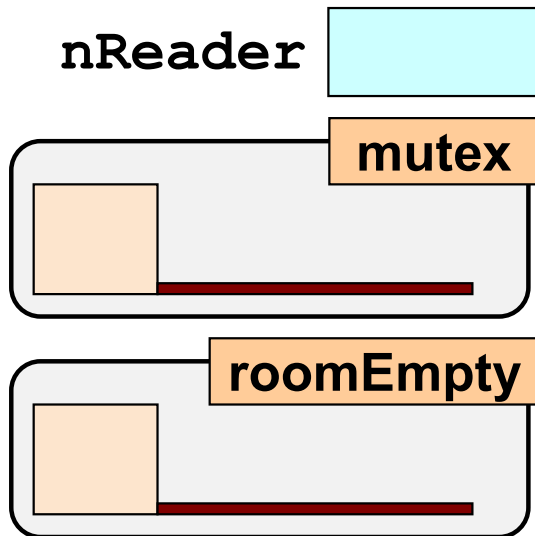
- Processes share a data structure **D**:
  - ❑ Reader: **Retrieves** information from **D**
  - ❑ Writer: **Modifies** information in **D**
- Writer must have exclusive access to **D**
- Reader can access with other readers



# Readers Writers: **Simple Version**

```
while (TRUE) {  
  
    wait( roomEmpty );  
  
    Modifies data  
  
    signal( roomEmpty );  
}
```

Writer



```
while (TRUE) {  
  
    wait( mutex );  
    nReader++;  
    if (nReader == 1)  
        wait( roomEmpty );  
    signal( mutex );  
  
    Reads data  
  
    wait( mutex );  
    nReader--;  
    if (nReader == 0)  
        signal( roomEmpty );  
    signal( mutex );  
}
```

Reader

---

# Readers Writers: **Evaluation**

- Convince yourself that the solution satisfies the specification
- However:
  - It has one problem
  - (hint: Something to do with writer....)

---

Ahem..... lab 3..... ahem.....

# **SYNCHRONIZATION IMPLEMENTATIONS**

# POSIX Semaphore

- Popular implementation of semaphore under Unix
- Header File:
  - ❑ `#include <semaphore.h>`
- Compilation Flag:
  - ❑ `gcc something.c -lrt`
  - ❑ Stand for "real time library"
- Basic Usage:
  - ❑ Initialize a semaphore
  - ❑ Perform `wait()` or `signal()` on semaphore

# *pthread* Mutex and Conditional Variables

- Synchronization mechanisms for pthreads
- Mutex (`pthread_mutex`):
  - ❑ Binary semaphore (i.e. equivalent Semaphore(1)).
  - ❑ Lock: `pthread_mutex_lock()`
  - ❑ Unlock: `pthread_mutex_unlock()`
- Conditional Variables( `pthread_cond` ):
  - ❑ Wait: `pthread_cond_wait()`
  - ❑ Signal: `pthread_cond_signal()`
  - ❑ Broadcast: `pthread_cond_broadcast()`

# Others

- Programming languages with thread support will have some forms of synchronization mechanism
- Examples:
  - ❑ **Java**: all object has built-in lock (mutex), **synchronized** method access, etc
  - ❑ **Python**: supports mutex, semaphore, conditional variable, etc
  - ❑ **C++**: Added built-in thread in C++11; Support mutex, conditional variable

# Summary

## ■ Synchronization:

- ❑ Problem: Race condition
- ❑ Solution: Critical Section
- ❑ Criteria of good solution:
  - Mutual Exclusion, progress, bounded waiting time, independence
- ❑ Important High Level Construct: Semaphore

## ■ Classic Synchronization problems:

- ❑ Producer + Consumer
- ❑ Reader + Writer
- ❑ Dining Philosophers

# Reference

- Modern Operating System (3<sup>rd</sup> Edition)
  - Chapter 2.4
- Operating System Concepts (7<sup>th</sup> Edition)
  - Chapter 5
- Edgar W. Dijkstra, “Note No.123: Cooperating Sequential Processes”
  - <http://www.cs.utexas.edu/users/EWD/ewd01xx/EWD123.PDF>